

Relation Algebra

§15. Dynamic Programming

§15.1. Theory

$h : FB \rightarrow B$ a map, $T : FA \rightarrow A$ an F-algebra, $R : B \rightarrow B$.

(15.1a)

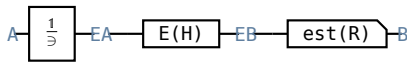
$H \triangleq (T)^\circ (h) : A \rightarrow B$, $M \triangleq \frac{H}{\subseteq} \text{est}(R)$ the problem to be solved, $(\mu X : G(X))$ as in $\triangleq \mu$.

$\frac{H}{\subseteq} \text{est}(R) \triangleq (\mu X : \frac{T^\circ}{\subseteq} \text{thin}(Q) P(F(X)h) \text{est}(R))$

(15.1b)

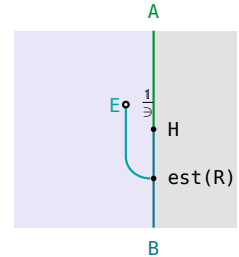
an optimum over everything H returns is reached by taking the input apart every way T allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

circuit



the problem to be solved, $H \triangleq (T)^\circ (h) \triangleq H$

Hinze–Marsden



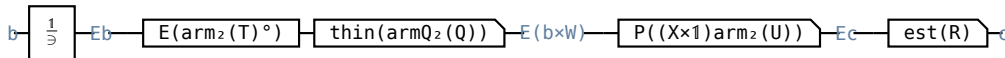
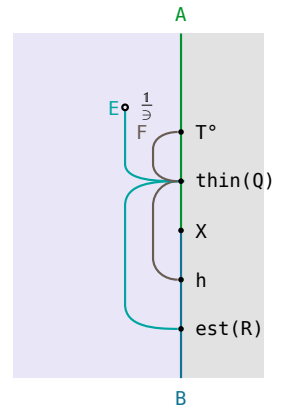
h monotonic on R and Q a preorder with $QF(H)h \subseteq F(H)hR$; $\text{thin}(Q)$ as in thin-laws. This is the same with the thinning step dropped — $1 \subseteq \text{thin}(Q)$, so the body and with it the fixed point only shrink.

□

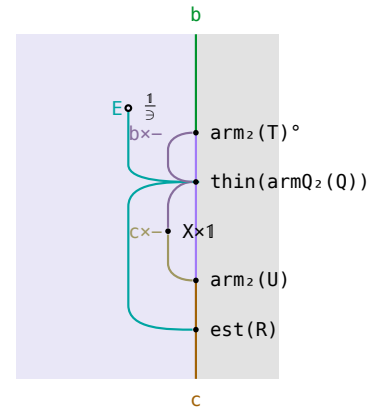
Knaster–Tarski leaves (9.1) $\frac{T^\circ}{\subseteq} P(F(M)h)$

$\text{est}(R) \subseteq M$; $M = H \cap (H^\circ \setminus R^\circ)$ splits that into (9.2) $\frac{T^\circ}{\subseteq} P(F(M)h) \text{est}(R) \subseteq H$ and (9.3) $H^\circ \frac{T^\circ}{\subseteq}$

$P(F(M)h) \text{est}(R) \subseteq R^\circ$, and both use only (9.4) = (7.10) $P(X) \text{est}(R) \subseteq (\exists X) \cap (E \setminus (XR^\circ))$ — est-710



= Proposition 9.1 at $T = [V_1, V_2]$, $h = [U_1, U_2]$, $Q = Q_1 + Q_2$, $V_2 V_1^\circ = 1$: FA is usually a coproduct, and disjoint ranges split the fixed point into one branch per summand. The fixed point is unique and entire — T° followed by F 's membership relation inductive, $\frac{T^\circ}{\subseteq}$ finite and non-empty, R connected



the condition	how it is discharged	(15.1c)
$F(R)h \subseteq hR$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{ cost} = F(\text{cost})k$, $F(\leq)k \subseteq k \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$	
$F(R \cap (H^\circ H))h \subseteq hR$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{ cost} = F(\langle \text{cost}, H^\circ \rangle)k$, $F(\leq \times 1)k \subseteq k \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from	
$QF(H)h \subseteq F(H)hR$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R)h \subseteq hR$, $VH \subseteq HR$	both conditions at once, split along the two arguments of a bifunctor	

§15.2. The string edit problem

$Op ::= \text{cpy Char} | \text{del Char} | \text{ins Char}$, $F(A, B) = 1 + (A \times B)$, $\alpha \triangleq [\text{nil}, \text{cons}]$.

(15.2a)

$\text{edit} \triangleq ([\text{base}, \text{step}]) : [Op] \rightarrow [\text{Char}] \times [\text{Char}]$, base returning $([], [])$, step (cpy $a, (xs, ys)$) = $([a] \# xs, [a] \# ys)$, step (del $a, (xs, ys)$) = $([a] \# xs, ys)$, step (ins $a, (xs, ys)$) = $(xs, [a] \# ys)$.

$\text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}])$, $R \triangleq \text{length} \leq \text{length}^\circ$, $U \triangleq T$, $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, $Q \triangleq 1 + (U \times V)$, empty the coreflexive on (xs, ys) with both lists empty.

unstep implements $\frac{\text{step}^\circ}{\exists} \text{thin}(U \times V) : \text{unstep} ([a] \# xs, []) = [(\text{del } a, (xs, []))], \text{unstep} ([], [b] \# ys) = [(\text{ins } b, ([], ys))], \text{unstep} ([a] \# xs, [b] \# ys) = (a \rightarrow b \rightarrow [(\text{cpy } a, (xs, ys))], [(\text{del } a, (xs, [b] \# ys)), (\text{ins } b, ([a] \# xs, ys))])$.

(15.2b)

$\frac{\text{edit}^\circ}{\exists} \text{est}(R) \equiv \text{mle}, \text{mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R))$ a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed	
circuit 	Hinze–Marsden
at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \alpha R$ is Proposition 9.2 at $\text{length} \triangleq ([\text{zero}, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R. The thinning condition $QF(1, \text{edit}^\circ) \alpha F(1, \text{edit}^\circ) \alpha R$ over $F(\text{Op}, [\text{Char}] \times [\text{Char}])$ is Proposition 9.4 at $U \triangleq T$ — $F(T, R) \alpha \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, — $\text{edit} (1 \times \text{suffix}) \sqsubseteq R^\circ \text{edit}$, $\text{edit} (\text{suffix} \times 1) \sqsubseteq R^\circ \text{edit}$. $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del, never lengthening the sequence	
Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty, the coreflexive on (xs, ys) with both lists empty, is where base returns	
unstep implements $\frac{\text{step}^\circ}{\exists} \text{thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and $\text{minlist}(R)$ implements $\text{est}(R)$. The same subproblem is solved many times over, so the running time is exponential in the two lengths	

(15.2c)

$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys)), \text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$
 $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)]), \text{fstcol} = \text{list}(\text{del}) \text{tails}$
 the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left
 $\text{column}(xs)([b] \# ys) = \text{nextcol}(xs)(b, \text{column}(xs)(ys))$
 $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)])(xus), xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$
 each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right
 $\text{base}(b, u) = [[\text{ins}(b)] \# u]$
 $= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [(\text{cpy}(a)] \# v) \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$
 $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§15.3. Optimal bracketing

tree A ::= tip A | bin (tree A, tree A), $FX = A + X^2$, so $F(R) = 1 + R^2$; $h \triangleq [\text{tip}, \text{bin}]$, $\text{flatten} \triangleq ([\text{wrap}, \text{cat}])^\circ$: tree A $\rightarrow$ list⁺ A, $H = \text{flatten}^\circ$.

$(\text{cost}, \text{size}) \triangleq ([\text{opt}, \text{opb}])^\circ$, $\text{opt} \triangleq (\text{zero}, \text{st})$, $\text{opb} ((cx, sx), (cy, sy)) = (\text{cb} (sx, sy) + cx + cy, \text{sb} (sx, sy))$.

sb associative, so $\text{size} = \text{flatten} \text{ sz}$ for a map $\text{sz} : R \triangleq \text{cost} \leq \text{cost}^\circ$, $g \triangleq [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi_1]$, single the coreflexive on singleton lists.

$\text{splits} \triangleq (\text{inits}^+, \text{tails}^+)$ zip, an implementation of $\frac{\text{cat}^\circ}{\text{g}}$; $\text{array} \triangleq \text{inits list}(\text{row})$, $\text{row} \triangleq \text{tails list}(\text{mct})$, $\text{col} \triangleq \text{inits list}(\text{mct})$.

$\text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R)$, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$.

(15.3a)

$\frac{\text{flatten}^\circ}{\text{g}}$ $\text{est}(R) \ni \text{mct}$, $\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix})$

a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

(15.3b)

circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{\frac{1}{\text{g}}} E(\text{list}^+(A)) \xrightarrow{E(\text{flatten}^\circ)} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ the specification — $\triangleq \text{flatten}$	
$\text{list}^+(A) \xrightarrow{\frac{1}{\text{g}}} E(\text{list}^+(A)) \xrightarrow{E([\text{wrap}, \text{cat}]^\circ)} E(\text{list}^+(A)) \xrightarrow{P([\text{tip}, (X \times X) \text{bin}])} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ split the list in every way, bracket both halves, join. The condition is monotonicity in context, $F(R) \ni (\text{flatten} \text{ flatten}^\circ) h \in hR$ — only trees with the same flattening are compared — which is Proposition 9.3 at $H^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip}, \text{bin}]$ $\text{cost} = (1 + (\text{cost}, \text{flatten})^2) g$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) g \sqsubseteq g \sqsubseteq (g \text{ monotonic on } \leq \text{ in its two cost arguments})$	
$\text{list}^+(A) \xrightarrow{\frac{1}{\text{g}}} E(\text{list}^+(A)) \xrightarrow{E(\text{cat}^\circ)} E(\text{list}^+(A)^2) \xrightarrow{P((X \times X) \text{bin})} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ = Proposition 9.1: wrap and cat have disjoint ranges, and single is the coreflexive on singleton lists, where wrap returns	
$\text{list}^+(A) \xrightarrow{\text{splits}} [\text{list}^+(A)^2] \xrightarrow{\text{list}((\text{mct} \times \text{mct}) \text{bin})} [\text{tree}(A)] \xrightarrow{\text{minlist}(R)} \text{tree}(A)$ $\text{splits} \triangleq (\text{inits}^+, \text{tails}^+)$ zip implements $\frac{\text{cat}^\circ}{\text{g}}$ and $\text{minlist}(R)$ implements $\text{est}(R)$. Exponential, since the segments of one list overlap	
$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}) \quad (9.7)$ = the tabulation: mct xs is needed for every non-empty segment xS, so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row})$, $\text{row} \triangleq \text{tails list}(\text{mct})$, $\text{col} \triangleq \text{inits list}(\text{mct})$, $\text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R)$	
$\text{col} = (\text{single} \rightarrow \text{head tip wrap}, (\text{init col}, \text{tail row}) \text{ next}) \quad (9.8)$ $\text{cons col} = (1 \times \text{array}) \text{ process} \quad (9.9)$ = $\text{row} = (\text{single} \rightarrow \text{head tip wrap}, (\text{mct}, \text{tail row}) \text{ cons}) \quad (9.10)$ a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$	
$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$ = $\text{addcol} \triangleq (\pi_1 \text{ tip wrap}, \text{step}) \text{ cons}$, $\text{step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$ the program: one fold building the array column by column, cubic in the length of the input	

§15.4. Data compression

$[A] ::= \text{nil} \mid \text{snoc } ([A], A), \text{ list}^+ A ::= \text{wrap } A \mid \text{snoc } (\text{list}^+ A, A), \text{ String} = [\text{Char}], \text{ Code} ::= \text{sym } \text{Char} \mid \text{ptr } (\text{String}, \text{String}^+), F(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}), \alpha \triangleq [\text{nil}, \text{snoc}].$

(15.4a)

$\text{decode} \triangleq ([\text{nil}, \text{extend}])^\circ : [\text{Code}] \rightarrow \text{String}, \text{extend } (xs, \text{sym } a) = xs \# [a], \text{extend } (xs, \text{ptr } (ys, zs)) = xs \# zs \text{ when } ys \# zs \text{ is a proper prefix of } xs \# zs; H = \text{decode}^\circ.$

$\text{size} \triangleq ([\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus}])^\circ$ with c, p the constant costs of a symbol and a pointer; $R \triangleq \text{size} \leq \text{size}^\circ$, $Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers.

$\text{lrt } ws = \text{est}(\text{prefix}^\circ \times (\tau + \tau)) \{(xs, (ys, zs)) \mid xs \# zs = ws, ys \# zs \text{ a proper prefix of } ws\}$, the longest repeated tail; $\text{reduce } (ws \# [a]) = (zs \neq [] \rightarrow [(ws, \text{sym } a), (xs, \text{ptr } (ys, zs))], [(ws, \text{sym } a)])$ with $(xs, (ys, zs)) = \text{lrt } (ws \# [a])$.

$\frac{\text{decode}^\circ}{\supset} \text{est}(R) \ni \text{encode}, \text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce } \text{list}((\text{encode} \times 1) \text{snoc}) \text{minlist}(R))$

(15.4b)

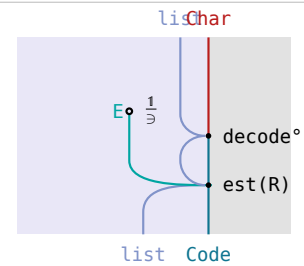
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit

$[\text{Char}] \xrightarrow{\frac{1}{\supset}} E[\text{Char}] \xrightarrow{E(\text{decode}^\circ)} E[\text{Code}] \xrightarrow{\text{est}(R)} [\text{Code}]$

the specification — $\triangleq \text{decode}$

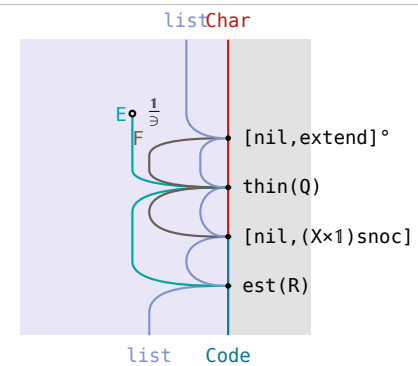
Hinze–Marsden



$[\text{Char}] \xrightarrow{\frac{1}{\supset}} E[\text{Char}] \xrightarrow{E([\text{nil}, \text{extend}]^\circ)} \text{thin}(Q) \xrightarrow{EF[\text{Char}]} P([\text{nil}, (X \times 1) \text{snoc}]) \xrightarrow{E[\text{Code}]} \text{est}(R) \xrightarrow{[\text{Code}]}$

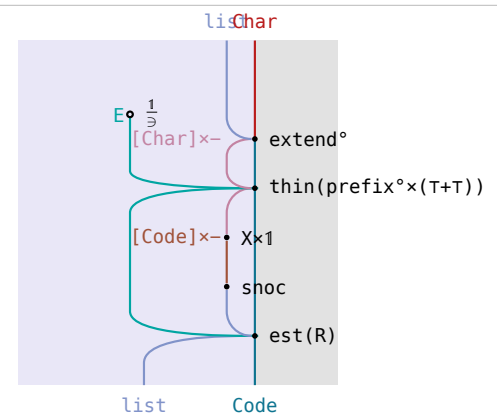
at $Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers. Monotonicity $F(R) \alpha \alpha R$ is routine, the two costs being constants: $(\tau + \tau)[c, p] = [c, p]$.

Proposition 9.4 splits the thinning condition in two — $F(\tau + \tau, R) \alpha \alpha R$, and $\text{decode } \text{prefix} \in R^\circ \text{ decode}$, for which $\text{decode } \text{init} \in R^\circ \text{ decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



$[\text{Char}] \xrightarrow{\frac{1}{\supset}} E[\text{Char}] \xrightarrow{E(\text{extend}^\circ)} \text{thin}(\text{prefix}^\circ \times (\tau + \tau)) \xrightarrow{E([\text{Char}] \times \text{Code})} P((X \times 1) \text{snoc}) \xrightarrow{E[\text{Code}]} \text{est}(R) \xrightarrow{[\text{Code}]}$

Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{\supset} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



$[\text{Char}] \xrightarrow{\text{reduce}} [[\text{Char}] \times \text{Code}] \xrightarrow{\text{list}((\text{encode} \times 1) \text{snoc})} [[\text{Code}]] \xrightarrow{\text{minlist}(R)} [\text{Code}]$

reduce implements $\frac{\text{extend}^\circ}{\supset} \text{thin}(\text{prefix}^\circ \times (\tau + \tau))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it

